MACHINE LEARNING AND DEEP LEARNING

Lab 02: RNNs & LSTMs

Dr. Giacomo Lucato, a.y. 2025-2026

RECAP

- Last time we saw how to perform an **Image Classification Task.**
- Best model for images: **Convolutional Neural Network** (CNN).
- Structure of a CNN:
  - **Convolutional Layers:** learnable filters to extract features;
  - **Max Pooling Layers:** fixed filters to reduce data dimensionality;
  - **Fully Connected Layers:** layers to perform the final prediction.
- Loss function: **Cross Entropy,** which assigns a **heavy penalty** whenever the model is **very confident in a wrong prediction**, a null penalty if the model is very confident in a right prediction, and scaled penalties according to intermediate combinations.

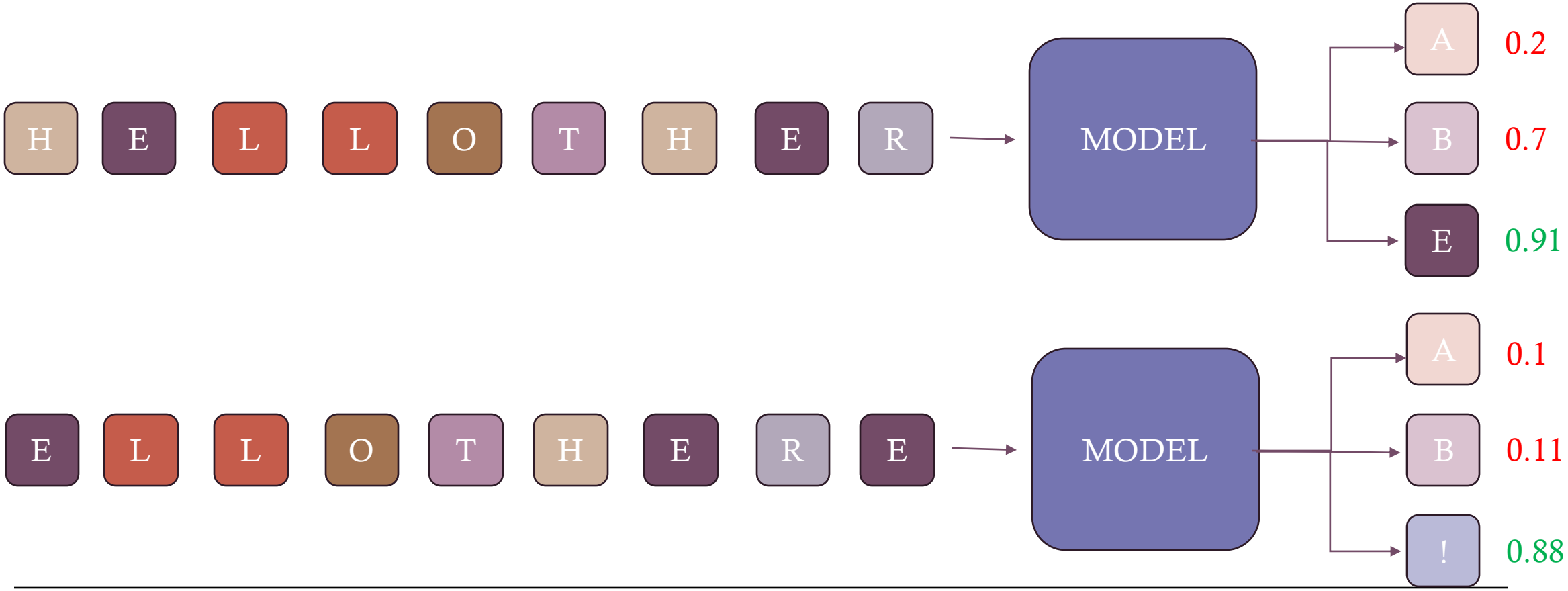
FORECASTING

- **Forecasting** is a ML task that assumes that **order of data points** matters.
- There are different types of forecasting:
  - **One-Step-Ahead Forecasting:** predicting only the vary next value of a sequence.
  - **Multi-Step Forecasting:** predicting a whole horizon of future values.
  - **Univariate vs Multivariate:** the first refers to prediction based on a single variable; the second to prediction based on multiple variables.
  - **Discrete vs Continuous:** it refers to data type.
- We will focus on **Univariate, Discrete One-Step-Ahead Forecasting,** but everything we will see can be applied without effort to other variants of forecasting.

ONE-STEP-AHEAD FORECASTING

- **One-Step-Ahead Forecasting** involves **predicting the very next token** of an input sequence.
- The task is carried on as follows:
  - **Input Window:** the model receives a sequence of N elements.
  - **Single Prediction:** the model outputs a probability distribution for the next character.
  - **The Shift:** Once the prediction is made, we "shift" our focus forward by one character, incorporating the new data to predict the next step after that.
- Forecasting is a **Regression task** when dealing with **numbers** (e.g., predicting tomorrow temperature) or a **Classification task** when dealing with **discrete tokens** (e.g., predicting the next word in a sentence).

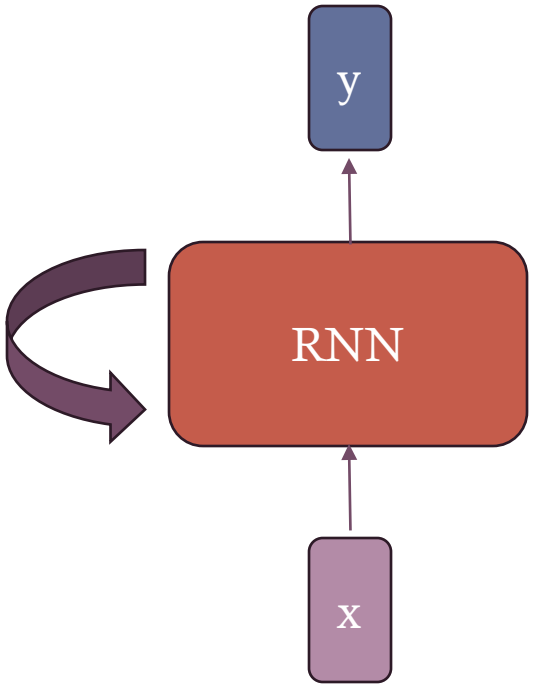
ONE-STEP-AHEAD FORECASTING - EXAMPLE



RECURRENT NEURAL NETWORK (RNN)

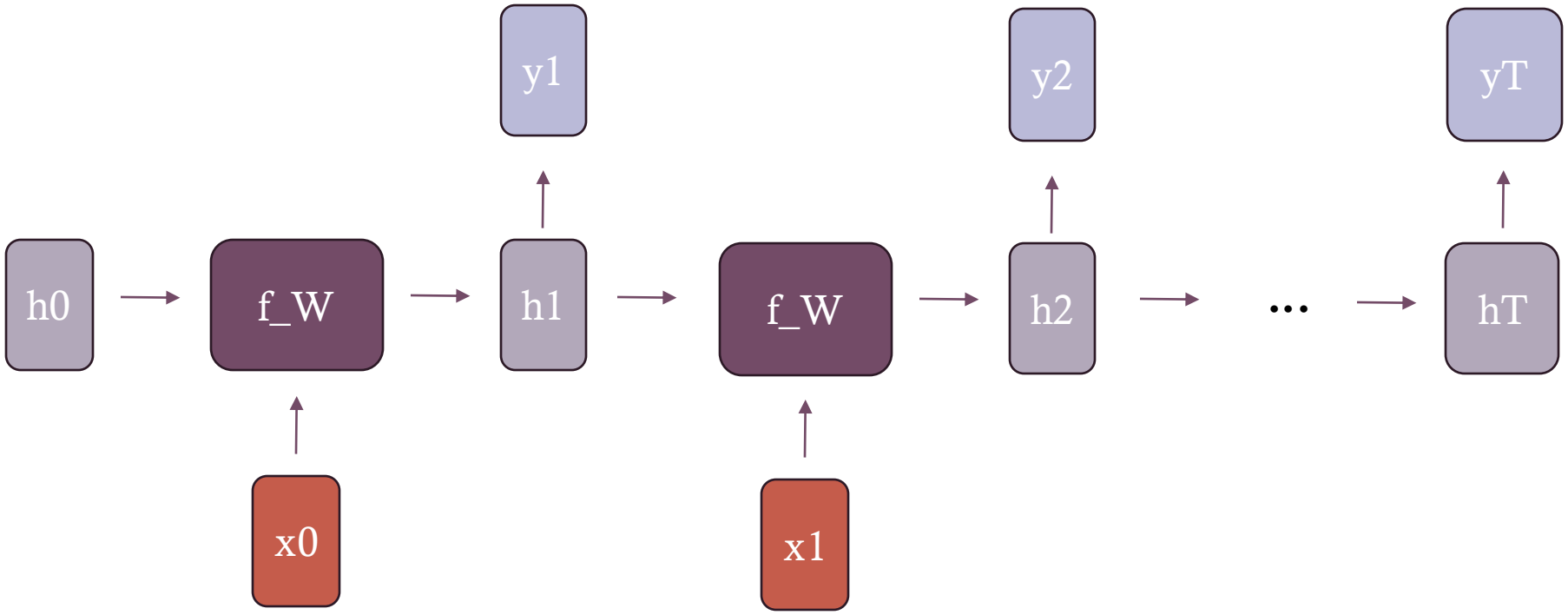
- **Recurrent Neural Networks (RNNs)** are a type of Artificial Neural Network specifically designed to handle **sequential data.**
- RNNs have **memory** of previous steps, that influence the current output.
- **Hidden State:** the hidden state (h) is the core of the RNN. As the model processes the input sequence, it updates this hidden state, that contains a sort of summary of every token of the sequence processed so far.
- At each time step, the network will have:
  - x_t: the input at time t (a single token).
  - h_t: the hidden state at time t, computed using x_t and h_t-1.
  - y_t: the prediction at time t (next element of the sequence).

RECURRENT NEURAL NETWORK (RNN) - STRUCTURE



- $h_t = f_W(h_{t-1}, x_t)$
- $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$
- $y_t = W_{hy}h_t$
- We have **three** functional sets of weights (W_{xh}, W_{hh}, W_{hy}). Because they are reused at every time step, the model learns patterns that are **invariant to time,** meaning it can recognize a word whether it appears at the beginning, middle, or end of a sentence.

RECURRENT NEURAL NETWORK (RNN) – UNROLLING THE STRUCTURE



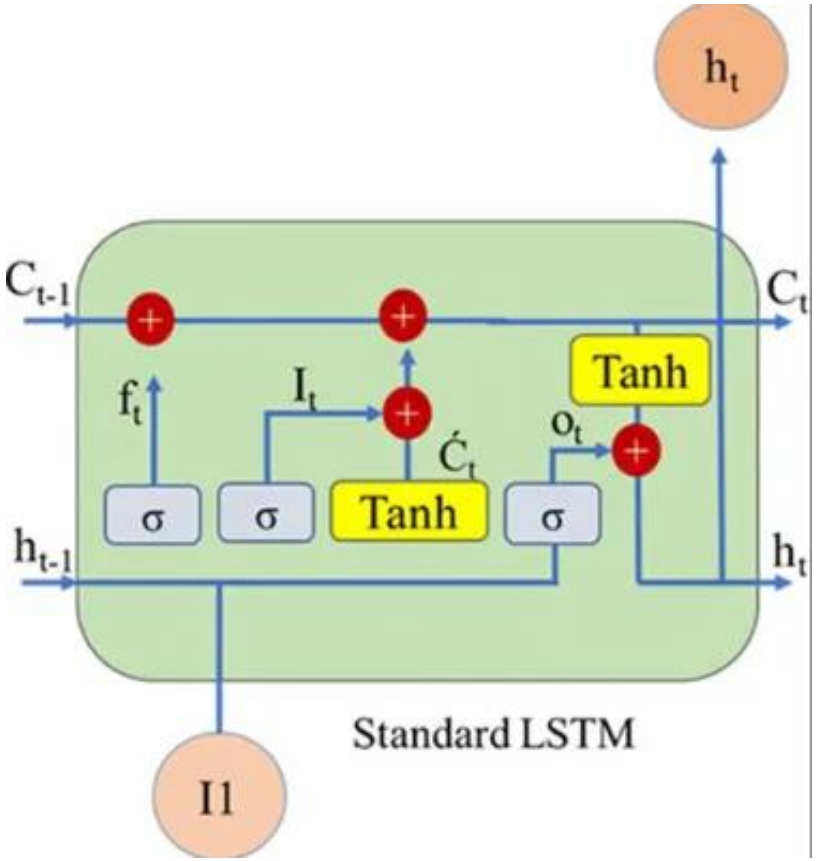
RECURRENT NEURAL NETWORK (RNN) - PROBLEMS

- The training of a RNN is carried on through a version of **backpropagation**, as it happens for other networks (MLPs, CNNs, …).
- Backpropagation is a process involving **computing a loss function** between the predicted target and the ground truth and **adjusting the weights** of the network to **reduce this loss.**
- For RNNs, since they use the same set of weights for each time step, backpropagation must start at the end of the time sequence and reach the beginning of it (Backpropagation Through Time).
- In this process the gradients are **multiplicated a lot of times,** and in long sequences this leads to **two problems:**
  - **Vanishing Gradients:** if weights are small, the gradients tend to disappear once the beginning of the sequence is reached. The model forgets early elements.
  - **Exploding Gradients:** if weights are large, gradients tend to become huge, crashing the training process.
- **Consequence:** RNN are unsuitable for long sequences.

LONG SHORT-TERM MEMORY (LSTM)

- **LSTM** is another type of Neural Network **designed to handle sequential data**, and it tries to solve the problems related to RNNs introducing a new element: the **cell state.**
- This cell state stores **long-term information**, and the LSTM can decide to add, delete or store information in this cell.
- In addition, LSTM has **gates,** that manage the cell state:
  - **Forget Gate:** decides which information from the past is no longer useful.
  - **Input Gate:** decides which new information should be added to the cell.
  - **Output Gate:** decides which information should be used to update the hidden state.
- Since the update of the cell state is **additive,** gradients can flow backwards without vanishing.

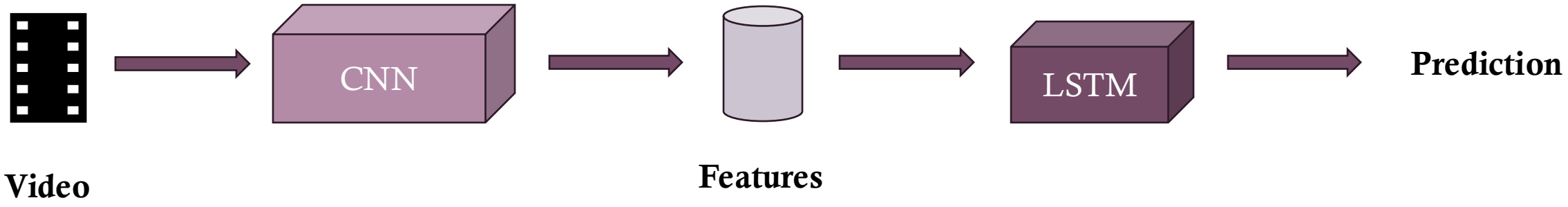
LONG SHORT-TERM MEMORY (LSTM)



VIDEO CLASSIFICATION

- **Videos** are an interesting type of data, since they are **sequences of images.**
- We studied how to handle images, and how to handle sequential data.
- **CNN** is the neural network used to deal with **images**, and **LSTM** is the one used for **sequential data.**
- To handle videos, we merge these two models:
  - **CNN:** is used to **extract features** from each frame.
  - **LSTM:** is used to capture the **time dependency** between frames.

VIDEO CLASSIFICATION



EXERCISE 1

- The goal of this exercise is to **train an LSTM** model to talk like a **Shakespeare's character.**
- Specifically, you should train the model to **predict the next character** of the input sequence, using a fixed-size window, shifted by one at every time step.
- **Assignment:** complete *Shakespeare.ipynb*, following the example *next_word_prediction.ipynb* if needed.

EXERCISE 2

- The goal of this exercise is to solve a **Video Classification Task.**
- Specifically, you should construct and train a model able to distinguish between two classes, where elements are videos.
- Given that videos are sequences of images, the most suitable model is a **combination** of a **CNN for feature extraction** and an **LSTM** for capturing the **temporal information.**
- **Assignment:** complete *video_classification.ipynb*, following the instructions provided in the comments.